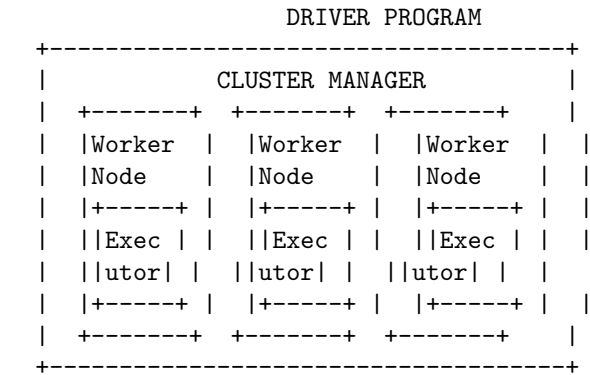


SPARK ARCHITECTURE OVERVIEW



RDD (Resilient Distributed Dataset)

- **Immutable** distributed objects
- **Fault-tolerant** with lineage
- **In-memory** computations
- **Lazy** evaluation

```
// Creating RDDs
val rdd1 = sc.textFile("path/to/file")
val rdd2 = sc.parallelize(List(1,2,3,4,5))
val rdd3 = sc.range(1, 100, 2)
```

- **Distributed** collection of data in rows with named columns
- **Optimized** with Catalyst optimizer
- **Supports** SQL queries
- **Schema-aware**

```
// Creating DataFrames
val df1 = spark.read.json("path/to/json")
val df2 = spark.read.csv("path/to/csv")
val df3 = Seq((1, "Alice"), (2, "Bob")).toDF("id", "name")
```

- **Type-safe** distributed data
- **Combines** RDD and DataFrame APIs
- **Compile-time** safety
- **Optimized** execution

```
case class Person(id: Int, name: String, age: Int)
val ds = Seq(Person(1,"Alice",25), Person(2,"Bob",30)).toDS()
```

OPERATIONS

Transformations (Lazy)

Operation	Description	Example
<code>map()</code>	1-to-1 transformation	<code>rdd.map(x => x * 2)</code>
<code>filter()</code>	Keep elements matching condition	<code>rdd.filter(x => x > 10)</code>
<code>flatMap()</code>	1-to-many transformation	<code>rdd.flatMap(x => x.split(" "))</code>
<code>groupByKey()</code>	Group by key	<code>pairRDD.groupByKey()</code>
<code>reduceByKey()</code>	Reduce by key	<code>pairRDD.reduceByKey(_ + _)</code>
<code>join()</code>	Join two RDDs	<code>rdd1.join(rdd2)</code>

Operation	Description	Example
<code>union()</code>	Combine two RDDs	<code>rdd1.union(rdd2)</code>
<code>distinct()</code>	Remove duplicates	<code>rdd.distinct()</code>

Actions (Eager)

Operation	Description	Example
<code>collect()</code>	Return all elements	<code>rdd.collect()</code>
<code>count()</code>	Count elements	<code>rdd.count()</code>
<code>first()</code>	First element	<code>rdd.first()</code>
<code>take(n)</code>	First n elements	<code>rdd.take(5)</code>
<code>reduce()</code>	Aggregate elements	<code>rdd.reduce(_ + _)</code>
<code>saveAsTextFile()</code>	Save to text file	<code>rdd.saveAsTextFile("path")</code>
<code>foreach()</code>	Apply function to each	<code>rdd.foreach(println)</code>

KEY CONCEPTS

Lazy Evaluation

- **Transformations** are not executed immediately
- **DAG** (Directed Acyclic Graph) is built
- **Actions** trigger actual computation
- **Optimization** happens at execution time

Caching & Persistence

```
// Cache levels
rdd.cache() // MEMORY_ONLY
rdd.persist(StorageLevel.MEMORY_ONLY)
rdd.persist(StorageLevel.MEMORY_AND_DISK)
rdd.persist(StorageLevel.DISK_ONLY)

// Unpersist
rdd.unpersist()
```

Partitioning

- **Default:** HashPartitioner
- **Custom:** RangePartitioner
- **Control:** `repartition()`, `coalesce()`
- **Check:** `rdd.partitions.size`

SPARK SQL

DataFrame Operations

```
-- SQL queries
df.createOrReplaceTempView("table")
spark.sql("SELECT * FROM table WHERE age > 21")

-- DataFrame API
df.select("name", "age")
df.filter($"age" > 21)
df.groupBy("department").agg(avg("salary"))
df.orderBy(desc("salary"))
```

Window Functions

```
import org.apache.spark.sql.expressions.Window
val windowSpec = Window.partitionBy("department").orderBy("salary")
df.withColumn("rank", rank().over(windowSpec))
```

SPARK MLlib

ML Pipeline

```
import org.apache.spark.ml.Pipeline
import org.apache.spark.ml.feature.{VectorAssembler, StringIndexer}
import org.apache.spark.ml.classification.LogisticRegression

// Feature processing
val assembler = new VectorAssembler()
  .setInputCols(Array("feature1", "feature2"))
  .setOutputCol("features")

val indexer = new StringIndexer()
  .setInputCol("label")
  .setOutputCol("indexedLabel")

// Model
val lr = new LogisticRegression()
  .setLabelCol("indexedLabel")
  .setFeaturesCol("features")

// Pipeline
val pipeline = new Pipeline()
  .setStages(Array(assembler, indexer, lr))

val model = pipeline.fit(trainingData)
val predictions = model.transform(testData)
```

Common Algorithms

- **Classification:** LogisticRegression, DecisionTreeClassifier, RandomForestClassifier
- **Regression:** LinearRegression, DecisionTreeRegressor, RandomForestRegressor
- **Clustering:** KMeans, GaussianMixture
- **Recommendation:** ALS (Alternating Least Squares)

SPARK STREAMING

DStreams

```
import org.apache.spark.streaming._

val ssc = new StreamingContext(sc, Seconds(1))
val lines = ssc.socketTextStream("localhost", 9999)

val words = lines.flatMap(_ .split(" "))
val wordCounts = words.map(word => (word, 1)).reduceByKey(_ + _)

wordCounts.print()
ssc.start()
ssc.awaitTermination()
```

Structured Streaming

```
import org.apache.spark.sql.streaming._

val df = spark.readStream
  .format("kafka")
  .option("kafka.bootstrap.servers", "host1:port1,host2:port2")
  .option("subscribe", "topic1")
  .load()

val result = df.groupBy("value").count()

val query = result.writeStream
```

```
.outputMode("complete")
.format("console")
.start()
```

```
query.awaitTermination()
```

DEPLOYMENT MODES

Local Mode

```
spark-submit --master local[*] --class MainClass app.jar
```

Standalone Cluster

```
# Start master
```

```
./sbin/start-master.sh
```

```
# Start workers
```

```
./sbin/start-slave.sh spark://master:7077
```

```
# Submit job
```

```
spark-submit --master spark://master:7077 --class MainClass app.jar
```

YARN

```
spark-submit \  
  --master yarn \  
  --deploy-mode cluster \  
  --class MainClass \  
  app.jar
```

Kubernetes

```
spark-submit \  
  --master k8s://https://k8s-master:6443 \  
  --deploy-mode cluster \  
  --class MainClass \  
  --conf spark.kubernetes.container.image=spark:latest \  
  app.jar
```

CONFIGURATION

Memory Settings

```
// Driver memory
```

```
spark-submit --driver-memory 4g
```

```
// Executor memory
```

```
spark-submit --executor-memory 4g
```

```
// Dynamic allocation
```

```
spark.dynamicAllocation.enabled = true
```

```
spark.dynamicAllocation.minExecutors = 1
```

```
spark.dynamicAllocation.maxExecutors = 10
```

Performance Tuning

- Parallelism: spark.default.parallelism
- Shuffle partitions: spark.sql.shuffle.partitions
- Serializer: spark.serializer = "org.apache.spark.serializer.KryoSerializer"
- Memory fractions: spark.memory.fraction

COMMON PATTERNS

Broadcast Variables

```
val broadcastVar = sc.broadcast(Array(1, 2, 3))
rdd.map(x => broadcastVar.value.contains(x))
```

Accumulators

```
val accum = sc.longAccumulator("My Accumulator")
rdd.foreach(x => accum.add(x))
println(accum.value)
```

Custom Partitioners

```
class CustomPartitioner extends Partitioner {
  def numPartitions = 10
  def getPartition(key: Any): Int = {
    // Custom logic
    key.hashCode % numPartitions
  }
}
```

USEFUL COMMANDS

Spark Shell

```
# Scala
spark-shell
```

```
# Python
pyspark
```

```
# R
sparkR
```

Submit Applications

```
spark-submit \
  --class com.example.MyApp \
  --master local[4] \
  --name "My Spark App" \
  --jars external.jar \
  myapp.jar
```

Monitor Jobs

- **Web UI:** <http://localhost:4040>
- **History Server:** <http://localhost:18080>
- **Metrics:** Prometheus/Grafana integration

BEST PRACTICES

Performance

- Use DataFrames over RDDs when possible
- Cache frequently used datasets
- Minimize shuffles with proper partitioning
- Use broadcast joins for small datasets
- Optimize data serialization

Memory Management

- Set appropriate executor memory
- Use off-heap memory for large datasets
- Monitor garbage collection

- Tune memory fractions

Code Organization

- Use case classes for structured data
- Implement proper error handling
- Log important operations
- Document custom functions

Created for quick reference and learning Apache Spark 3.x | Scala/Python APIs Date: November 2025